

Creating an XCode 2021 CSMP Examples Project

SKM 3/2/2021

Brief. Creates an Xcode C++ Command Line Tool in as much as an automated fashion as possible.

Step by step instructions

1. In XCode: Create new project, add files and configure

New project -> macOS -> Command Line Tool -> choose options you want for your project -> product name (organisation identifier your university) -> Language C++ -> create (do not create repository unless you want to copy all the source code into there)

Keep all your projects in a suitable high-level directory. This will ascertain simple paths to files (Xcode will create enclosing folders).

Xcode now built and opened a project file for you creating folder-like yellow “Groups” with the name of the project and the “Products”. The latter will contain the executable with the same default name. When you open this group it is shown in red because this file has yet to be created.

Type “run” (click triangular arrow at top to build), which will compile and link the default executable “Hello World” that is within the main file that Xcode created automatically for you.

Set the Project Format to Xcode 12.0-compatible or whatever the latest version is.

Within XCode’s Project Navigator

Remove main.cpp, replacing it with ExamplesMain.cpp

Right click and select “Move to the trash”

Right click CSMP Examples group -> Add -> Options -> find ExamplesMain.cpp in subdirectory examples of the csm-api-directory and add it to the project.

A reference to this file will now have been added to your project tree. Verify in the CSMP Examples project directory that no copy of this file was accidentally made.

Like you did for “Hello World”

Compile, run and remove variable_set_preprocessor.cpp main() from project

This is a program that creates `csm::INDEX` sets serving as C++ header files input to specific example applications, see CSMP User Guide. Two sets need to be generated with this program to support the CSMP examples:

```
VariableSet_TracerTransfer.h
VariableSet_TwoPhaseFlow.h
VariableSet_CO2GeoSequestration.h
```

Creating these sets involves running the **variable_set_preprocessor** from the command line, providing the corresponding MS Excel spreadsheets with the variables output to .csv text files as command line arguments. Just run the executable without arguments and the right syntax is shown. The required spreadsheet output files are stored in:

```
source/includes/integration/finite_volumes/generic_transport_scheme/
```

This is also the directory where you need to run the programme. Afterwards, to remove the program from the Xcode project just unselect it in the Target Membership box of XCode’s Project Inspector.

You can now add the generated header files to your project although they will be discovered automatically since they are already in the source code directory tree.

Compile, run and remove generate_number_types() main

Apply the same procedure to this program, running it in the

source/includes/model/

directory to produce the output file:

CSMP_number_types.h

No arguments are required. Taking into account your C++ compiler settings, this file will contain typedefs to generate correctly sized integral types for your CSMP program. For instance, double64 will be defined as such and distinct from double128. This guarantees that CSMP computations have the same accuracy across platforms, operating systems and compilers.

The number types program stems from a time when the C++ standards did not implement fixed size types yet and, strictly speaking, is superseded since C++17. Notwithstanding, the CSMP-specific types are retained to support legacy applications. Before running generate_number_types select “short enums” in the C++ compiler settings of XCode.

You can now add the generated header file to your project although it will be discovered automatically since it is already in a source code directory.

Within XCode's Project Navigator

Create group to contain all the source code of the examples

New group without folder -> give name -> Add files to -> “examples” -> select files as done for the main file under 2.

The main file will be greyed out as it already belongs to the project.

Use the default Xcode settings for the Add command, i.e. do not copy files, do not create groups here but just references.

Within XCode's Project Navigator

Add references the the CSMP API library' source code to project

Again by right-clicking the CSMP Examples group -> Add files to -> choose source/
And **select “Create Groups”** in the menu.

This is very important because it creates a group for each subfolder in the repository rather than drawing the entire directory structure including folder references into your project.

When completed correctly, CSMP's directory tree should now appear in XCode's folder / file directory with yellow groups matching the names of the folders in the CSMP repository.

Within XCode's Project Navigator

Add third-party libraries to project

The CSMP examples also draw on a range of Open Source libraries, some of which have however been modified to achieve compatibility with modern C++, such as the meschach++, IAPS (equation of state for water) and jpeg libraries. To make your life easier, these are therefore supplied together with the API library. In addition, there are interfaces to the Algebraic Multigrid for Systems (SAMG) linalg library from Fraunhofer, Germany. The latter stubs later allow Xcode to connect with the dynamic link library files at runtime. The required so files are also contained in subdirectories of third_party/. Further below, we will see what compilation flags are required to include these libraries and how dynamic linkage is facilitated via definition of a

DYLD_LIBRARY_PATH. SAMG also requires a license, which will become accessible via a VPN link to your university or over the campus network.

Importantly, not all files in the third_party/ directory are needed for the Xcode project. The solution is as easy as deleting them from the project again.

Delete (remove references to):

Catch library - everything apart from the folder single_include/
SAMG library - keep only the files that relate to the Mac OS.

Project configuration

If you now compile CSMP, you will get an error that CSMP_DEFAULT_LINEAR_SOLVER is an abstract data type. If you echo this type from the preprocessor you will get:

```
CSMP_DEFAULT_LINEAR_SOLVER          csmpp::Solver
CSMP_DEFAULT_LINEAR_SOLVER_SETTINGS csmpp::SolverSettings
```

which are purely virtual base classes. If you are unable to access a license to the SAMG_Solver, then define, for instance, csmpp::**GaussJordan_Solver** and its corresponding settings as solver defaults.

```
#undef CSMP_DEFAULT_LINEAR_SOLVER
#define CSMP_DEFAULT_LINEAR_SOLVER csmpp::GaussJordan_Solver
#undef CSMP_DEFAULT_LINEAR_SOLVER_SETTINGS
#define CSMP_DEFAULT_LINEAR_SOLVER_SETTINGS
csmpp::GaussJordan_SolverSettings
```

There will also be warnings which can be easily disabled by clicking the little (x) in the Filter window of the Buildtime output.

All these errors and warning result since you have not yet supplied Xcode with any CSMP-specific compilation flags. This is part of the next step, which is to configure your Xcode example project.

Within XCode's Project Navigator select the blue CSMP_Examples project and then go to the **Build Settings** central pane in the Xcode GUI. Here select **All** in the bar below which probably is set to **Basic**. This gives access to a million settings, but, fortunately, most defaults are just fine.

Architectures - probably you need only the one of the computer that you are building this project on.

Assets & Build Locations - leave as is.

Build Options - this is where you may later enable testability and profiling. Nothing to do here for the moment.

Deployment - you could pick a desired directory here to install your executable in. We will set that somewhere else later.

Headers, Kernel Module - leave as is.

Linking - only needs adjustments if the order of the dynamic link libraries is scrambled.

Localisation - nothing unless you want to translate CSMP into another language.

Packaging - you might change the name of your executable here.

Search Paths - if you are using SAMG, set the path here to the directory that contains the SAMG DLLs.

Library Search Paths - put /Users/ [your username] /Documents/CSMP_GIT/csmc-api-library/thirdparty/samg/macosx/

Signing - would be to authenticate your code.

Testing, Text-Based API, Versioning - nothing needs to be changed here.

Apple Clang - Address Sanitizer - nice functionality at a huge performance cost, but you should try to get confidence in the CSMP code.

Apple Clang - Custom Compiler Flags - we will use the current defaults.

Apple Clang - Language - CHANGE

C-Language Dialect - c11

Short Enumeration Constants - YES

Apple Clang - Language - C++ - CHANGE C++ Language Dialect to C++17

Which library you use here has a big impact on the debugging info and performance of the STL!

Apple Clang - Language - Modules - defaults are fine.

Apple Clang - Language - Objective C - we are not using this.

Apple Clang - Preprocessing - CSMP++ preprocessor macros need to be defined here.

Preprocessor Macros

Debug

```
DEBUG=1
PYRAMID_TRIANGULAR_FACETS
CSMP_WITH_SAMG_SOLVER
SAMG_UNIX_LINUX
SAMG_LCASE_USCORE
SAMG_MULTIPLE_INSTANCES
```

Release

```
NDEBUG=1
PYRAMID_TRIANGULAR_FACETS
CSMP_WITH_SAMG_SOLVER
SAMG_UNIX_LINUX
SAMG_LCASE_USCORE
SAMG_MULTIPLE_INSTANCES
```

Note that the SAMG-related pre-processor directives are not needed if another solver is used, but we strongly recommend SAMG. If no license is available for it, one can still run it as a single-level solver (CG etc. or even PARDISO will be available).

PYRAMID_TRIANGULAR_FACETS is an instruction to the finite volume scheme to break the warped quadrilateral FV-facets in the apex of the pyramid into planar triangles, see Paluszny et al. (2007, Geofluids). This increases accuracy at the price of a slight increase in computational cost.

If you want to use camp together with the CFD support library CNGS via the respective interface in the interfaces/ directory, you need to define

CSMP_WITH_CGNS_CFD_DATA_STANDARD_LIBRARY

And download this library from GitHub by yourself because we do not include it under third_party/ as it is still being developed and will work “out of the box.”

Apple Clang - Undefined Behaviour Sanitizer - Hopefully CSMP++ does not have this!

Apple Clang - Warning Policies - defaults are fine.

Apple Clang - Warnings - All languages

Turn warnings on for

- Missing Function Prototypes
- Suspicious Implicit Conversions
- Uninitialised Variables
- Unused Parameters

Apple Clang - Warnings - C++ - add a warning for Nonvirtual Destructors

Apple Clang - Warnings - Objective-C - we are not using these languages.

Apple Clang - Warnings - Objective-C and ARC

OSACompile - Build Options

Static Analyzer - Analysis Policy - the defaults for this cool tool are fine.

Static Analyzer - General Issues - the default is all analysis is one. This is fine.

Static Analyzer - Issues - Apple APIs - is not really of significance for the Command Line Tool we are building.

Static Analyzer - Issues - Objective-C - we are not using this language.

Static Analyzer - Issues - Security - use defaults.

User-Defined - no extras are needed here.

Note that the latest version of Xcode parses the DOxygen documentation in the CSMP++ library by default and parsing errors will be reported as warnings.

2. Compiling the CSMP API

Everything is set now, but dependent on the customised version of the API that you may want to create, some little issues might still trip up your compilation. Thus, for instance, included old versions of “*old.cpp” files etc. Just delete these obviously superseded source files.

3. Linking, including dynamic linkage

CSMP should now compile, but will get back to you with linker errors such as symbols or “”library not found. These are linker errors and may stem for sources that still need to be included or dynamic link libraries that XCode cannot find because the path has not been defined.

To set a path to the SAMG dynamic link libraries,

Find the XCode Scheme “CSMP Examples” at the top of your project and click it.



A new window will appear the software generation steps in the left column and the panel options on the top. It will be on “Info” where you can see that the current setting refers to the Debug version of your project (later we will learn how to duplicate the correctly configured scheme so that we can also build the release version of the software).

Choose the “Arguments” panel to define the necessary “Environment Variables”:

Click + and define DYLD_LIBRARY_PATH in the box on the left.

Find one of the SAMG supporting DLLs in the Finder, right-click it to reveal the option to copy the pathname to this file, and then paste this name into the right box to associated the dynamic library path variable with the corresponding path.

This copied path should look something like this:

```
/Users/smatthai/Documents/CSMP_GIT/csmc-api-library/thirdparty/samg/macosx/  
libamg_mult.dylib
```

Watch out that you do not include the library file itself!

While we are at it, also set the license file for SAMG, but only if you are using a node-locked license on the laptop. To do this, define LM_LICENSE_FILE to the corresponding file on your machine, wherever that lives.

Sometimes the linker also complains about duplicated files, especially duplicated main() functions.

4. Running the CSMP Examples

In order to run the CSMP Examples with the suggested input files, you need to create a target directory and place the fixtures and necessary data files (models etc.) into it.

As a first step, go back to the project “CSMP Examples” -> Edit Scheme

And have a look at the Options pane. For “Working Directory”, select “Use custom working directory”, navigating into the desired directory by clicking on the folder icon.

As explained in the CSMP UserGuide, each example comes with a member function called “Specifications”, which explains what input files it needs and what running options there are. Typically there is a specific variables file (text file: “...-variables.txt”) plus an input file set for the creation of the model, the selection of target regions for simulation (“...-regions.txt”), and for the configuration of the models (“...-configuration.txt”). While the model files reside in directories with their name, all textiles are stored under fixtures.

All input files are located in the subdirectory of the API /csmc-api-library/examples/data/

The examples project launches with a menu that allows the user to select from three groups of examples:

C++

Software Functionality

Simulation of Physical Processes

Read the source code of the examples to get specific explanations and guidance with them. Use the ExperimentalExample to try some of your own computations.

Have fun! - Stephan

Other Topics

(To be continued)

Creating debug and release versions of the project.

Profiling.